

# Wykład 5

## Działanie AVR reszta Wyświetlacze LED Wyświetlacze LCD

### AVR – budowa

#### **Boot loader**

Bootloader w mikrokontrolerach, to mini program zagnieżdżony w pewnym sektorze pamięci uC. Dzięki niemu możliwe jest wgrywanie programu z pominięciem programatora.

Rozwiązanie takie stosowane jest coraz częściej np.: dla ułatwienia aktualizacji oprogramowania

#### **Interfejs zewnętrznej pamięci danych**

W niektórych mikrokontrolerach istnieje interfejs zewnętrznej pamięci RAM jako przedłużenie wewnętrznej pamięci SRAM układu, który cechuje duża uniwersalność (RAM, flash, inne peryferia).

Po odpowiedniej konfiguracji porty wejścia i wyjścia przejmują funkcję linii adresowych danych i sterujących

## AVR – budowa

### Interfejsy programowania i uruchomieniowe

#### Interfejsy programowania

- Pamięć programu, EEPROM i bity sterujące układów ATmega mogą być programowane co najmniej przez 2 sposoby:
  1. Programowanie w systemie w którym układ pracuje (ISP – *In System Programming*) najczęściej przez interfejs szeregowy SPI
  2. Programowanie w specjalnym programatorze równoległym (bardziej skomplikowany, droższy, mający więcej możliwości)

#### Interfejsy uruchomieniowe (*debug interface*)

- JTAG i debugWIRE
- Służą do testowania i kontroli wewnętrznych podukładów mikrokontrolera z zewnątrz (mogą działać w trybie ISP)

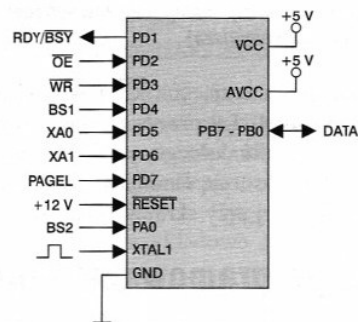


## AVR – budowa

### Interfejsy programowania i uruchomieniowe

#### Równoległy interfejs programowania

- Krótkie czasy zapisu i weryfikacji
- Można wszystko przeprogramować
- Umożliwia zapis **układu w którym bity sterujące niepoprawnie skonfigurowano**



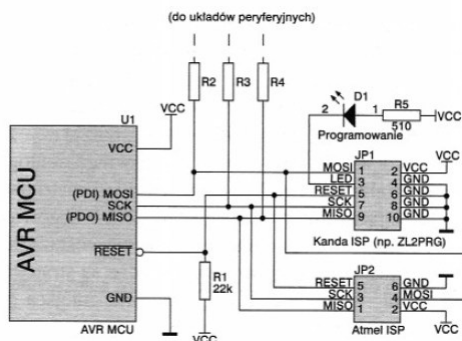
Linie sygnałowe wykorzystywane przy programowaniu równoległym

## AVR – budowa

### Interfejsy programowania i uruchomieniowe

Szeregowy interfejs programowania - SPI

- Najbardziej popularny
- Tryb ISP – w SPI używane są tylko trzy linie sygnałowe
- Wykorzystuje się końcówki które mogą być podłączone do urządzeń peryferyjnych więc trzeba zadbać by na czas programowania nie wpływały one na ten proces (zworki, rezystory ograniczające prąd pobierany)



Sposób dołączenia programatora ISP do uC w układzie docelowym

## AVR – budowa

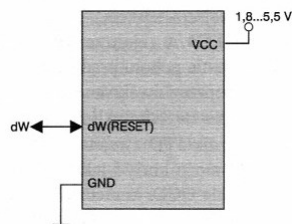
### Interfejsy programowania i uruchomieniowe

Interfejs JTAG

- Przydatny w pracach prototypowych
- Pozwala na znalezienie przerw w połączeniach z obwodem drukowanym
- Śledzenie wykonywanie programu
- Badanie stanu mikrokontrolera
- Programowanie pamięci

Interfejs debugWIRE

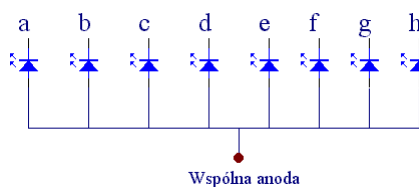
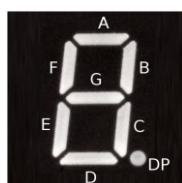
- Prostszy JTAG dla mniejszych uC np. ATtiny
- Np. Nie pozwala na znalezienie przerw w połączeniach z obwodem drukowanym



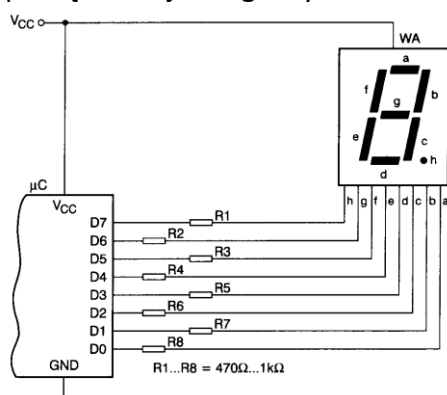
Sposób dołączenia debugWIRE do uC w układzie docelowym

# Multiplexowane wyświetlacze LED

## Wyświetlacz siedmiosegmentowy



Sposób podłączenie jednego wyświetlacza do mikrokontrolera



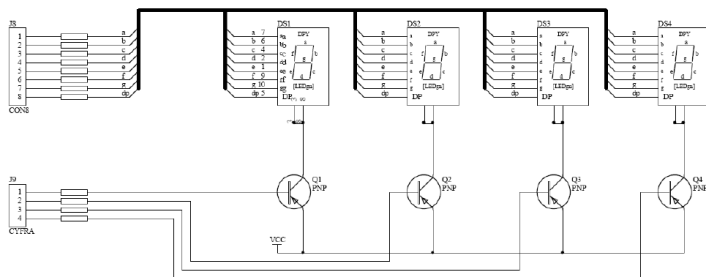
Sprawa wydajności prądowej!

## Multipleksowanie

Dla wyświetlania większej ilości cyfr, typowo 4, takie podłączenie jak na slajdzie poprzednio staje się za bardzo „portożerne”.

Dlatego rozwiązanie z multipleksowaniem, czyli cyklicznym załączaniu każdej cyfry z taką częstotliwością aby dla oka nie było widoczne migotanie. Doświadczalnie przyjęto taką częstotliwość na 50 Hz.

### Typowy układ podłączenia 4 wyświetlaczy siedmiosegmentowych

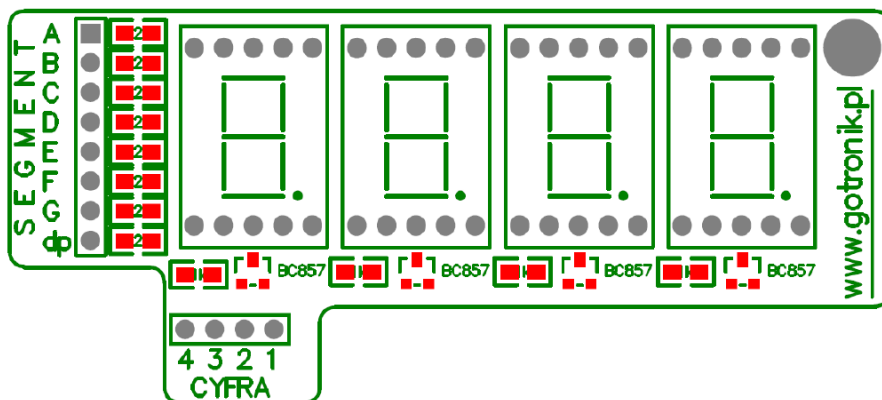


Wybór aktywnego wyświetlacza dokonuje się poprzez podanie 0 na bazy tranzystorów

Wybór aktywnych segmentów dokonuje się poprzez podanie 0 na wejście od **a** do **g** i **dp**

## Multipleksowanie

### Multipleksowane wyświetlacze 7 segmentowe LED



# Multiplexowanie

## Zasada działania programu obsługującego multiplexowane wyświetlanie na 4 siedmiosegmentowych wyświetlaczach.

Ma działać niezależnie od programu głównego

Należy stworzyć coś w rodzaju biblioteki, którą będzie można użyć w różnych projektach

Program bazujący na przerwaniach generowanych przez układ licznika który co 5 ms ( $1s/50\text{odświeżeń}/4\text{ segmenty}=200\text{ Hz}$ ), które będą po kolei odświeżały obraz na danym wyświetlaczu aktualną wartością przypisaną zmiennej

## Multiplexowanie

### Ustawianie licznika

8-bit  
Timer/Counter  
Register  
Description

Bit

Read/Write  
Initial Value

| 7    | 6     | 5     | 4     | 3     | 2    | 1    | 0    |       |
|------|-------|-------|-------|-------|------|------|------|-------|
| FOC0 | WGM00 | COM01 | COM00 | WGM01 | CS02 | CS01 | CS00 | TCCR0 |
| W    | R/W   | R/W   | R/W   | R/W   | R/W  | R/W  | R/W  |       |
| 0    | 0     | 0     | 0     | 0     | 0    | 0    | 0    |       |

### Ustawienie trybu pracy licznika

Table 38. Waveform Generation Mode Bit Description<sup>(1)</sup>

| Mode | WGM01 (CTC0) | WGM00 (PWM0) | Timer/Counter Mode of Operation | TOP  | Update of OCR0 | TOV0 Flag Set-on |
|------|--------------|--------------|---------------------------------|------|----------------|------------------|
| 0    | 0            | 0            | Normal                          | 0xFF | Immediate      | MAX              |
| 1    | 0            | 1            | PWM, Phase Correct              | 0xFF | TOP            | BOTTOM           |
| 2    | 1            | 0            | CTC                             | OCR0 | Immediate      | MAX              |
| 3    | 1            | 1            | Fast PWM                        | 0xFF | BOTTOM         | MAX              |

### Ustawienie podziału preskalera

Clear Timer on Compare match

Table 42. Clock Select Bit Description

| CS02 | CS01 | CS00 | Description                                             |
|------|------|------|---------------------------------------------------------|
| 0    | 0    | 0    | No clock source (Timer/Counter stopped).                |
| 0    | 0    | 1    | clk <sub>ICP</sub> (No prescaling)                      |
| 0    | 1    | 0    | clk <sub>ICP</sub> /8 (From prescaler)                  |
| 0    | 1    | 1    | clk <sub>ICP</sub> /64 (From prescaler)                 |
| 1    | 0    | 0    | clk <sub>ICP</sub> /256 (From prescaler)                |
| 1    | 0    | 1    | clk <sub>ICP</sub> /1024 (From prescaler)               |
| 1    | 1    | 0    | External clock source on T0 pin. Clock on falling edge. |
| 1    | 1    | 1    | External clock source on T0 pin. Clock on rising edge.  |

Rejestr do którego wpisujemy liczbę do której licznik ma zliczać w trybie CTC

$OCR0 = 16\text{MHz}/1024/200\text{Hz} = 78,125 \approx 78$   
(licznik osiągnie wartość 78 po ok. 5 ms)

### Ustawienie zezwolenia na wywołanie przerwań od trybu CTC timera0

| Bit           | 7     | 6     | 5      | 4      | 3      | 2     | 1     | 0     |       |
|---------------|-------|-------|--------|--------|--------|-------|-------|-------|-------|
|               | OCIE2 | TOIE2 | TICIE1 | OCIE1A | OCIE1B | TOIE1 | OCIE0 | TOIE0 | TIMSK |
| Read/Write    | R/W   | R/W   | R/W    | R/W    | R/W    | R/W   | R/W   | R/W   |       |
| Initial Value | 0     | 0     | 0      | 0      | 0      | 0     | 0     | 0     |       |

Bit 1 – OCIE0: Timer/Counter0 Output Compare Match Interrupt Enable

- Bit 0 – TOIE0: Timer/Counter0 Overflow Interrupt Enable

## Multiplexowanie

### Ustawianie licznika

```
TCCR0 |= (1<<WGM01);           // tryb CTC
TCCR0 |= (1<<CS02) | (1<<CS00); // preskalator = 1024
OCR0 = 78;                      // podział przez 78 (rej. porównania)
TIMSK |= (1<<OCIE0);           // zezwolenie na przerwanie Compare Match
```

### Przerwanie

```
sei(); // włączenie globalnego zezwolenia na przerwanie
```

Szkielet procedury obsługi przerwania

```
ISR( nazwa_przerwania )
{
    // ciało procedury obsługi przerwania
}
```

```
ISR(TIMER0_COMP_vect)
```

```
{
    // Włączanie po kolei wyświetlaczy (A) z odpowiednią
    // konfiguracją włączonych segmentów (B)
}
```

(A) Anody wyświetlaczy podłączone do 4 pierwszych linii portu D PDO – PD3

```
Przerwanie1 - 1110
Przerwanie2 - 1101
Przerwanie3 - 1011
Przerwanie4 - 0111
```

Krążące zero

```
Przerwanie5 - 1110
Przerwanie6 - 1101
Przerwanie7 - 1011
Przerwanie8 - 0111
```

## Multiplexowanie

### Przerwanie

Najpierw krążąca jedynka w pętli 0001 -> 0010 -> 0100 -> 1000 -> 0001

```
uint8_t licznik=1;

while(1)
{
    licznik <= 1;
    if(licznik>8) licznik=1;
    PORTA = licznik;
    _delay_ms(1000);
}
```

Tu powinno być PORTD

A tak krążące zero w pętli

```
uint8_t licznik=1;

while(1)
{
    licznik <= 1;
    if(licznik>8) licznik=1;
    PORTA = ~licznik;
    _delay_ms(1000);
}
```

Taki schemat możemy przenieść do przerwania, z tym że licznik musi zostać zadeklarowany jako zmienna **static**, oraz w tym momencie zerujemy 4 pozostałe linie portu D – PD4-PD7, dlatego stosujemy **maskowanie wartości**, które wygląda następująco:

```
ANODY_PORT = ( ANODY_PORT & 0xF0 ) | ( ~licznik & 0x0F );
```

|                                      |           |                 |
|--------------------------------------|-----------|-----------------|
| Wartość portu A przed modyfikacją:   | 0110 1110 | <- początek     |
| maska bitowa 0xF0:                   | 1111 0000 | <- operacja AND |
| wynik1:                              | 0110 0000 |                 |
| modyfikacja zmiennej licznik o wart: | 1111 1101 |                 |
| maska bitowa 0x0F:                   | 0000 1111 | <- operacja AND |
| wynik2:                              | 0000 1101 |                 |
| -----                                |           |                 |
| wynik1:                              | 0110 0000 |                 |
| wynik2:                              | 0000 1101 | <- operacja OR  |
| Wartość do zapisania do PORTU ^:     | 0110 1101 | <----- koniec   |

## Multipleksowanie

### Przerwanie

#### (B) Katody wyświetlaczy, podłączone do portu A

Trzeba wykonać tablicę wzorców cyfr, które będą wyświetlane

Najlepiej w pamięci FLASH (parametr PROGMEM), bo jest niezmienna i nie musi zajmować miejsca w RAM

```
// definicja tablicy zawierającej definicje bitowe cyfr LED
const uint8_t cyfry[15] PROGMEM = {
    ~(SEG_A|SEG_B|SEG_C|SEG_D|SEG_E|SEG_F), // 0
    ~(SEG_B|SEG_C), // 1
    ~(SEG_A|SEG_B|SEG_D|SEG_E|SEG_G), // 2
    ~(SEG_A|SEG_B|SEG_C|SEG_D|SEG_G), // 3
    ~(SEG_B|SEG_C|SEG_F|SEG_G), // 4
    ~(SEG_A|SEG_C|SEG_D|SEG_F|SEG_G), // 5
    ~(SEG_A|SEG_C|SEG_D|SEG_E|SEG_F|SEG_G), // 6
    ~(SEG_A|SEG_B|SEG_C|SEG_F), // 7
    ~(SEG_A|SEG_B|SEG_C|SEG_D|SEG_E|SEG_F|SEG_G), // 8
    ~(SEG_A|SEG_B|SEG_C|SEG_D|SEG_F|SEG_G), // 9
    0xFF // NIC (puste miejsce)
};

// definicje bitów dla poszczególnych segmentów LED
#define SEG_A (1<<0)
#define SEG_B (1<<1)
#define SEG_C (1<<2)
#define SEG_D (1<<3)
#define SEG_E (1<<4)
#define SEG_F (1<<5)
#define SEG_G (1<<6)
#define SEG_DP (1<<7)

#define NIC 10
```

## Multipleksowanie

### Przerwanie

#### Kompletna procedura obsługi przerwania

```
ISR(TIMER0_COMP_vect)
{
    static uint8_t licznik=1; // zmienna do przełączania kolejno anod wyświetlacza

    ANODY_PORT = (ANODY_PORT | MASKA_ANODY); // wygaszenie wszystkich wyświetlaczy

    if(licznik==1) LED_DATA = pgm_read_byte( &cyfry[cy1] ); // gdy zapalony wyświetl.1 podaj stan zmiennej c1
    else if(licznik==2) LED_DATA = pgm_read_byte( &cyfry[cy2] ); // gdy zapalony wyświetl.2 podaj stan zmiennej c2
    else if(licznik==4) LED_DATA = pgm_read_byte( &cyfry[cy3] ); // gdy zapalony wyświetl.3 podaj stan zmiennej c3
    else if(licznik==8) LED_DATA = pgm_read_byte( &cyfry[cy4] ); // gdy zapalony wyświetl.4 podaj stan zmiennej c4

    ANODY_PORT = (ANODY_PORT & ~MASKA_ANODY) | (~licznik & MASKA_ANODY); // cykliczne przełączanie kolejnej
    // operacje cyklicznego przesuwania bitu zapalającego anody w zmiennej licznik anody w każdym przerwaniu
    licznik <= 1; // przesunięcie zawartości bitów licznika o 1 w lewo
    if(licznik>8) licznik = 1; // jeśli licznik większy niż 8 to ustaw na 1
}
```



# Wyświetlacze LCD

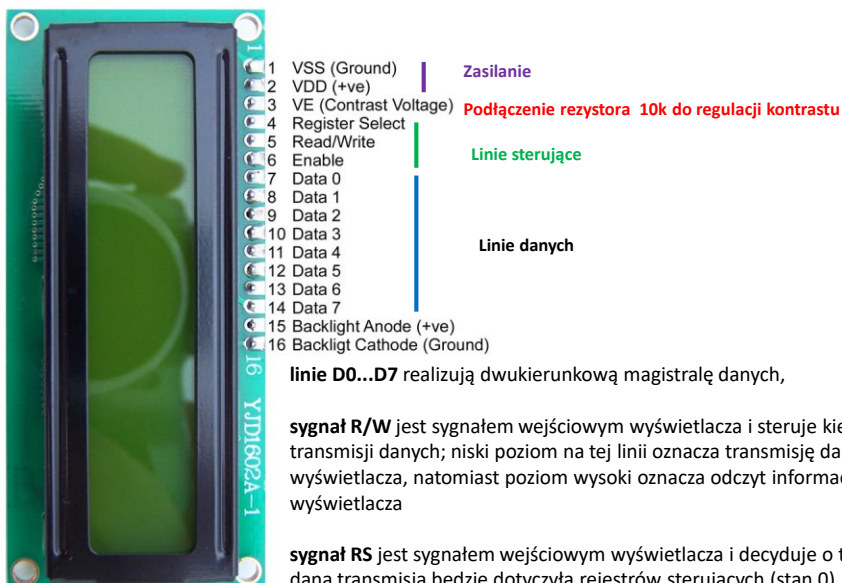
Wyświetlacz LCD (Liquid Crystal Display) wyświetlacz ciekłokrystaliczny



Najbardziej popularny wyświetlacz 2x16 linii ze sterownikiem HD 44780



## Wyświetlacz LCD z sterownikiem HD44780



linie D0...D7 realizują dwukierunkową magistralę danych,

**sygnał R/W** jest sygnałem wejściowym wyświetlacza i steruje kierunkiem transmisji danych; niski poziom na tej linii oznacza transmisję danych do wyświetlacza, natomiast poziom wysoki oznacza odczyt informacji z wyświetlacza

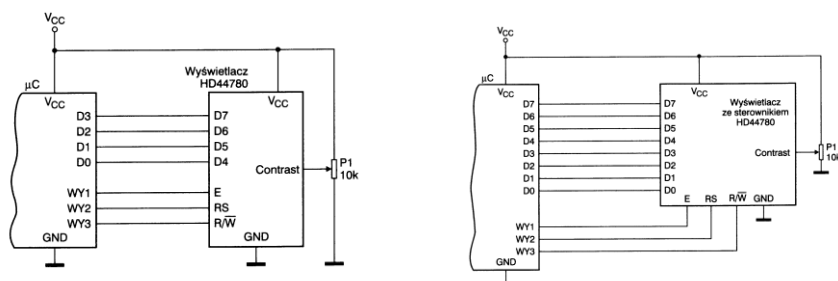
**sygnał RS** jest sygnałem wejściowym wyświetlacza i decyduje o tym, czy dana transmisja będzie dotyczyła rejestrów sterujących (stan 0), czy też transmitowane będą dane przeznaczone do wyświetlenia (stan 1);

**sygnał E** jest sygnałem strobującym/wyzwalającym.

## Wyświetlacz LCD z sterownikiem HD44780

Możliwe sterowanie z wykorzystaniem 4 lub 8 linii danych

- 4 linie – typowe aplikacje (mniejsza liczba wykorzystanych portów)
- 8 linii – tam gdzie korzystamy z 8 bitowej magistrali danych z adresowaniem



W trybie 8-bitowym wszystkie dane są przesyłane w trakcie jednego cyklu

w trybie 4-bitowym do przesłania informacji potrzeba dwóch cykli transmisji: najpierw dla bardziej znaczącej, a następnie dla mniej znaczącej połówki bajtu za pomocą linii D4...D7

## Wyświetlacz LCD z sterownikiem HD44780

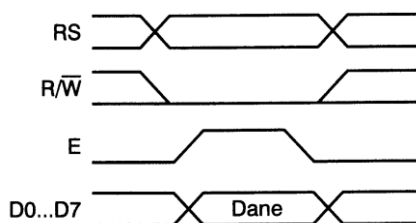
| Reżym               | RS | RW | D7                      | D6                            | D5                  | D4 | D3 | D2 | D1  | D0 | Funkcja                                          |
|---------------------|----|----|-------------------------|-------------------------------|---------------------|----|----|----|-----|----|--------------------------------------------------|
| Clear display       | 0  | 0  | 0                       | 0                             | 0                   | 0  | 0  | 0  | 0   | 1  | Czyszczenie wyświetlacza                         |
| Cursor home         | 0  | 0  | 0                       | 0                             | 0                   | 0  | 0  | 0  | 1   | x  | Ust. kursora w poz 0,0                           |
| Entry mode set      | 0  | 0  | 0                       | 0                             | 0                   | 0  | 0  | 1  | I/D | S  | Określenie trybu pracy kursora/okna wyświetlacza |
| Display On/Off      | 0  | 0  | 0                       | 0                             | 0                   | 0  | 1  | D  | C   | B  | Wł/Wył wyświetlacza                              |
| Cursor/displ. shift | 0  | 0  | 0                       | 0                             | 0                   | 1  | S  | R  | x   | x  | Przesuwanie kursora                              |
| Function set        | 0  | 0  | 0                       | 0                             | 1                   | D  | N  | F  | x   | x  | Tryb pracy wyświetlacza                          |
| CGRAM set           | 0  | 0  | 0                       | 1                             | Adres pamięci CGRAM |    |    |    |     |    | Ustawia adr. w CGRAM                             |
| DDRAM set           | 0  | 0  | 1                       | Adres pamięci DDRAM           |                     |    |    |    |     |    | Ustawia adr. w DDRAM                             |
| Busy flag read      | 0  | 1  | B                       | Adres pamięci DDRAM lub CGRAM |                     |    |    |    |     |    | Odczyt flagi zajętości                           |
| Data write          | 1  | 0  | Zapisywany bajt danych  |                               |                     |    |    |    |     |    | Zapis znaków                                     |
| Data read           | 1  | 1  | Odczytywany bajt danych |                               |                     |    |    |    |     |    | Odczyt znaków                                    |

*I/D = 1: kursor lub okno wyświetlacza przesuwają się w prawo  
I/D = 0: kursor lub okno wyświetlacza przesuwają się w lewo  
S = 1: po wpisaniu znaku do LCD kursor stoi w miejscu a przesuwają się zawartość okna  
S = 0: po wpisaniu znaku do LCD przesuwają się kursor; zawartość okna pozostaje  
D = 1/0: włączenie/wyłączenie wyświetlacza  
C = 1/0: włączenie/wyłączenie kursora  
B = 1/0: włączenie/wyłączenie migania kursora  
R = 1/0: kierunek przesuwu kursora w prawo/lewo  
D = 1: interfejs 8-bitowy  
D = 0: interfejs 4-bitowy  
N = 1: wyświetlacz dwunierszowy  
N = 0: wyświetlacz jednolierszowy  
F = 1: rozmiar znaku 5x10 punktów  
F = 0: rozmiar znaku 5x7 punktów*

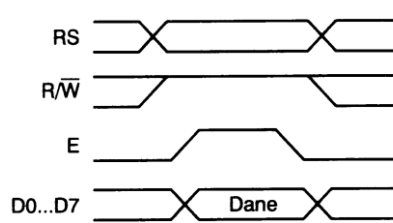
Zbiór poleceń sterownika

## Wyświetlacz LCD z sterownikiem HD44780

Cykl zapisu danych do wyświetlacza



Cykl odczytu danych do wyświetlacza

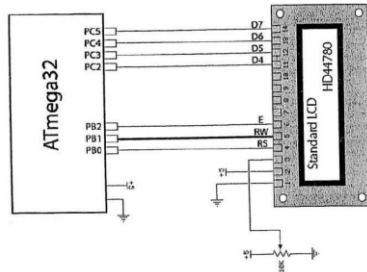


Przy wykonywaniu obu operacji linie sterujące RS i R/W muszą przyjąć odpowiednie stany zanim na linii E zostanie wygenerowany impuls. Podczas wysyłania danych do wyświetlacza o zatrzaśnięciu przesyłanej informacji decyduje zboczne opadające sygnału E, natomiast przy odczycie stabilny stan szyny danych występuje dla E=1.

Istnieją konkretne zależności czasowe długości impulsów i odstępów między nimi żeby wyświetlacz poprawnie działał.

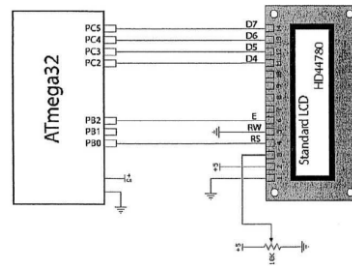
## Wyświetlacz LCD z sterownikiem HD44780

### Sterowanie wyświetlaczem LCD z linią RW podpiętą do uC



- Umożliwia szybsze wysyłanie danych do wyświetlacza
- Odczytuje się znacznik zajętości wyświetlacza

### Sterowanie wyświetlaczem LCD z linią RW na stałe połączoną z masą



- Wolniejsze wyświetlanie, ale w typowych sytuacjach w pełni wystarczające
- Oszczędza się 1 linię uC

## Wyświetlacz LCD z sterownikiem HD44780

### Programowanie obsługi wyświetlacza

#### Dwie drogi:

1. Stworzenie oprogramowania od początku
2. Wykorzystanie jednej z wielu dostępnych bibliotek
  - Korzystamy z bibliotek przedstawionych w książce M. Kardasia – *Mikrokontrolery AVR. Język C podstawy programowania*
  - `// rozdzielczość wyświetlacza LCD (wiersze/kolumny)`
  - `#define LCD_ROWS 2 // ilość wierszy wyświetlacza LCD`
  - `#define LCD_COLS 16 // ilość kolumn wyświetlacza LCD`
  - 
  - `// tu ustalamy za pomocą zera lub jedynki czy sterujemy pinem RW`
  - `// 0 - pin RW podłączony na stałe do GND`
  - `// 1 - pin RW podłączony do mikrokontrolera`
  - `#define USE_RW 0`

## Wyświetlacz LCD z sterownikiem HD44780

### Programowanie obsługi wyświetlacza

#### Konfiguracja sprzętu

```
// tu konfigurujemy port i piny do jakich podłączymy linie D7..D4 LCD
#define LCD_D7PORT A
#define LCD_D7 6
#define LCD_D6PORT A
#define LCD_D6 5
#define LCD_D5PORT A
#define LCD_D5 4
#define LCD_D4PORT A
#define LCD_D4 3|
// tu definiujemy piny procesora do których podłączamy sygnały RS,RW, E
#define LCD_RS_PORT A
#define LCD_RS 0

#define LCD_RWPORT A
#define LCD_RW 1

#define LCD_EPORT A
#define LCD_E 2
```

## Wyświetlacz LCD z sterownikiem HD44780

### Opis dostępnych funkcji

```
void lcd_init(void) // INICJALIZACJA WYŚWIETLACZA LCD
void lcd_blink_off(void) // Wyłącza miganie kursora na LCD
void lcd_blink_on(void) // Włącza miganie kursora na LCD
void lcd_cursor_off(void) // Wyłączenie kursora na LCD
void lcd_cursor_on(void) // Włączenie kursora na LCD
void lcd_home(void) // Powrót kursora na początek
void lcd_cls(void) // Kasowanie ekranu wyświetlacza
void lcd_locate(uint8_t y, uint8_t x) //Ustawienie kursora w pozycji Y-wiersz, X-kolumna
void lcd_defchar(uint8_t nr, uint8_t *def_znak) // Definicja własnego znaku na LCD z pamięci RAM
void lcd_defchar_E(uint8_t nr, uint8_t *def_znak) // Definicja własnego znaku na LCD z pamięci EEPROM
void lcd_defchar_P(uint8_t nr, const uint8_t *def_znak) // Definicja własnego znaku na LCD z pamięci FLASH
void lcd_int(int val) // Wyświetla liczbę dziesiętną na wyświetlaczu LCD
void lcd_hex(uint32_t val) // Wyświetla liczbę szesnastkową HEX na wyświetlaczu LCD
void lcd_str_E(char * str) // Wysłanie stringa do wyświetlacza LCD z pamięci EEPROM
void lcd_str_P(const char * str) // Wysłanie stringa do wyświetlacza LCD z pamięci FLASH
void lcd_str(char * str) // Wysłanie stringa do wyświetlacza LCD z pamięci RAM
void lcd_char(char c) // Wysłanie pojedynczego znaku do wyświetlacza LCD w postaci argumentu
```

## Wyświetlacz LCD z sterownikiem HD44780

### 2 rodzaje pamięci

**Pamięć DD-RAM** (Display Data RAM) – pamięć danych w której przechowywane są kody ASCII wyświetlanych znaków

**Pamięć CG-RAM** (Character Generator RAM) ma pojemność 64 bajtów i pozwala na zdefiniowanie do ośmiu znaków użytkownika (dowolne kombinacje matrycy 5x7(8)). Takie znaki otrzymują kody od 0 do 7 i mogą być wyświetlone podobnie jak znaki zdefiniowane w CG-ROM.

#### Definiowanie znaku

| Projektowany znak | Matryca znaku                                                                     | Bajty w CGRAM                                                                                | Adres w CGRAM                                                | Adres znaku w DDRAM |
|-------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------|
| Ł                 |  | 00010000<br>00010000<br>00010010<br>00010100<br>00011000<br>00010000<br>00011111<br>00000000 | 0x00<br>0x01<br>0x02<br>0x03<br>0x04<br>0x05<br>0x06<br>0x07 | 0x00                |
| Ż                 |  | 00000010<br>00000100<br>00011111<br>00000010<br>00000100<br>00001000<br>00011111<br>00000000 | 0x08<br>0x09<br>0x0A<br>0x0B<br>0x0C<br>0x0D<br>0x0E<br>0x0F | 0x01                |
| ą                 |  | 00000000<br>00000000<br>00001110<br>00000001<br>00001111<br>00010001<br>00001111<br>00000000 | 0x10<br>0x11<br>0x12<br>0x13<br>0x14<br>0x15<br>0x16<br>0x17 | 0x02                |

```
uint8_t tabl[] = { 0,0,14,1,15,17,15,2 }; // litera ą
lcd_defchar( 0x80, tabl );
lcd_str("m"x80"czny"); // napis „mączny”
```

<http://www.bastek79.com/2012/03/hd44780-generator-wlasnych-znakow/>